

矩陣式鍵盤的控制

如果按鍵較多，已經不太可能每隻腳都接一個按鍵，這樣會很浪費，比較常見的作法，都是採用矩陣型的接法，再利用掃描的方式，逐行掃描，看那一行有按鍵被壓下，再利用程式判斷，解出對應的按鍵值。

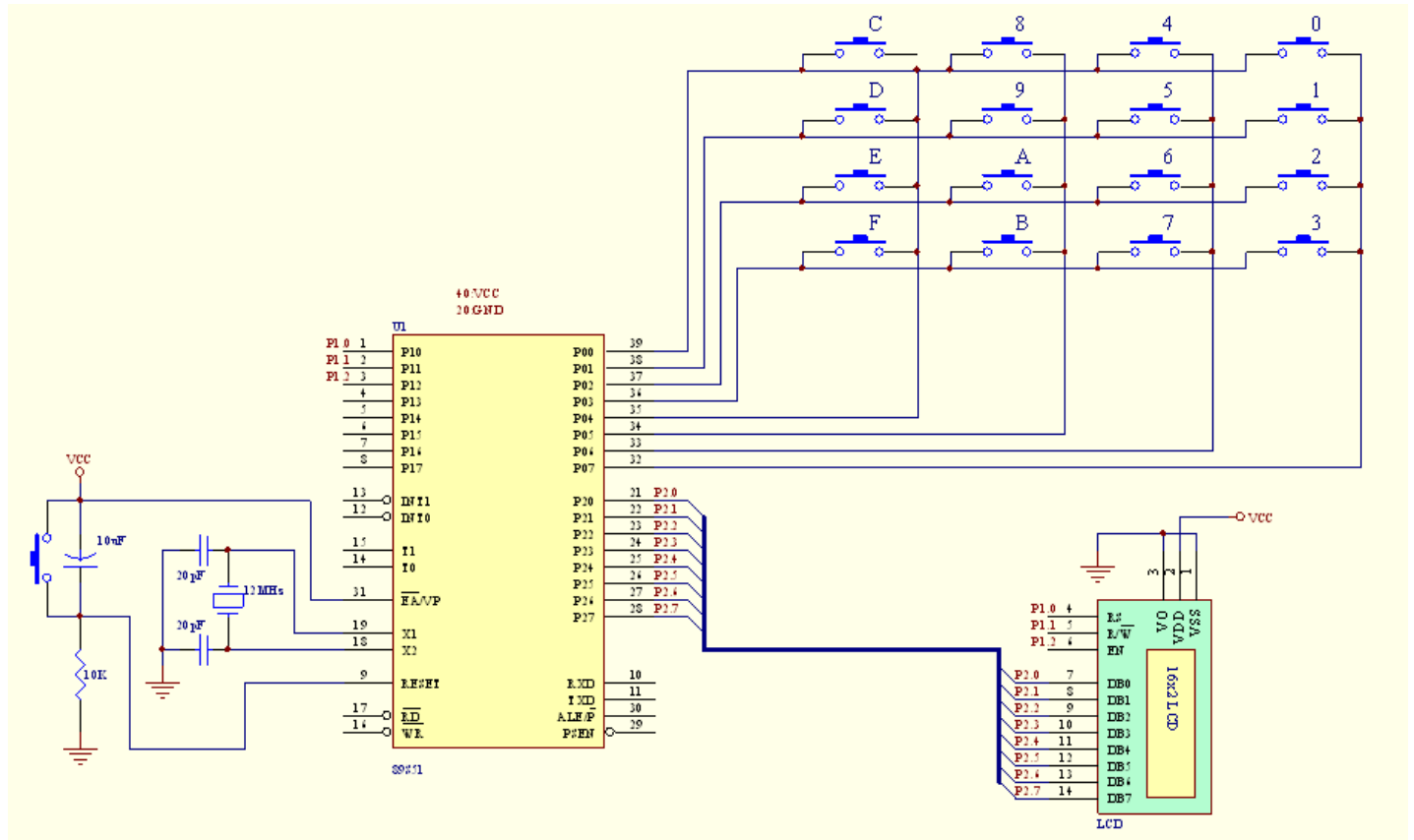


圖 3. 矩陣式鍵盤控制電路圖(未接提升電阻)

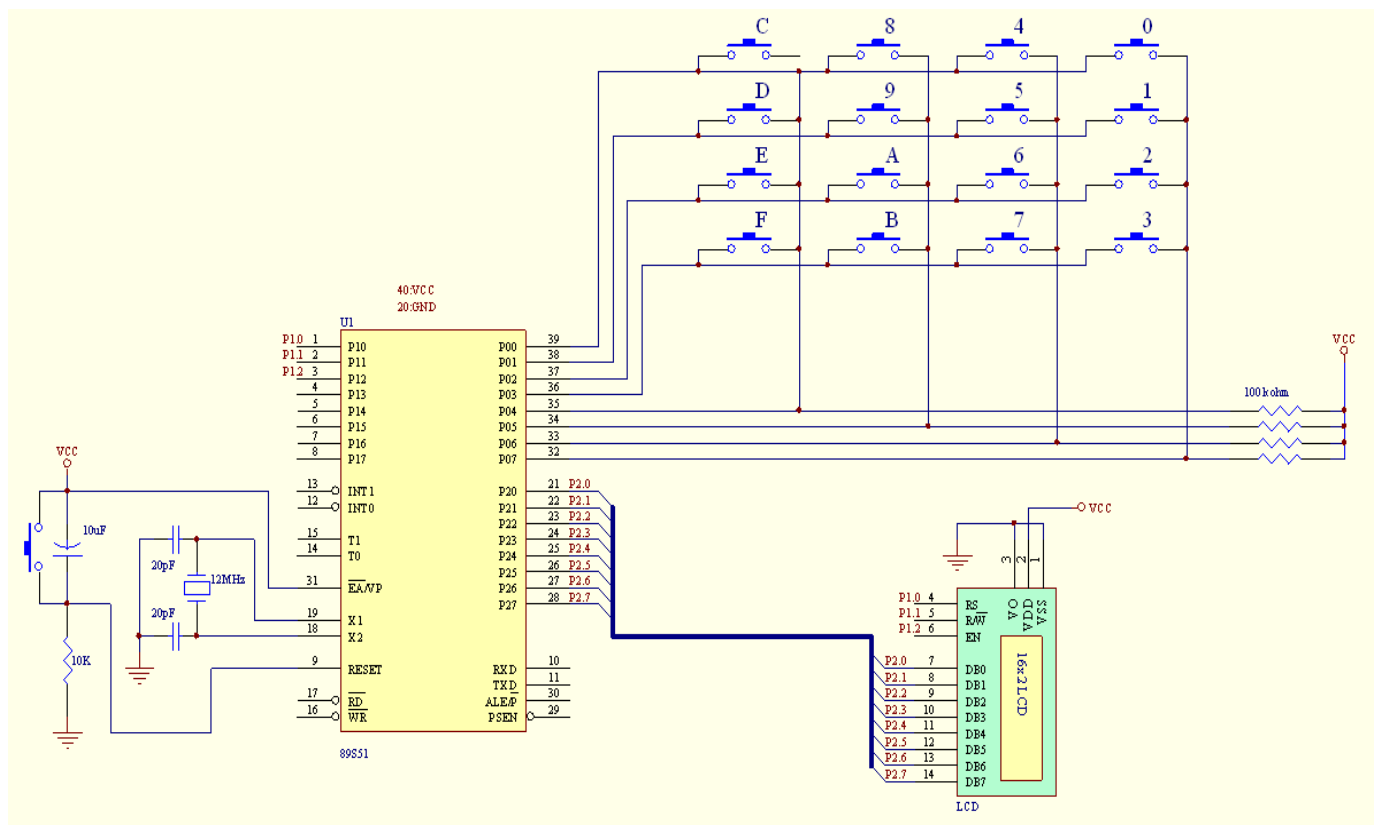


圖 4. 矩陣式鍵盤控制電路圖(外加提升電阻，提供電流)

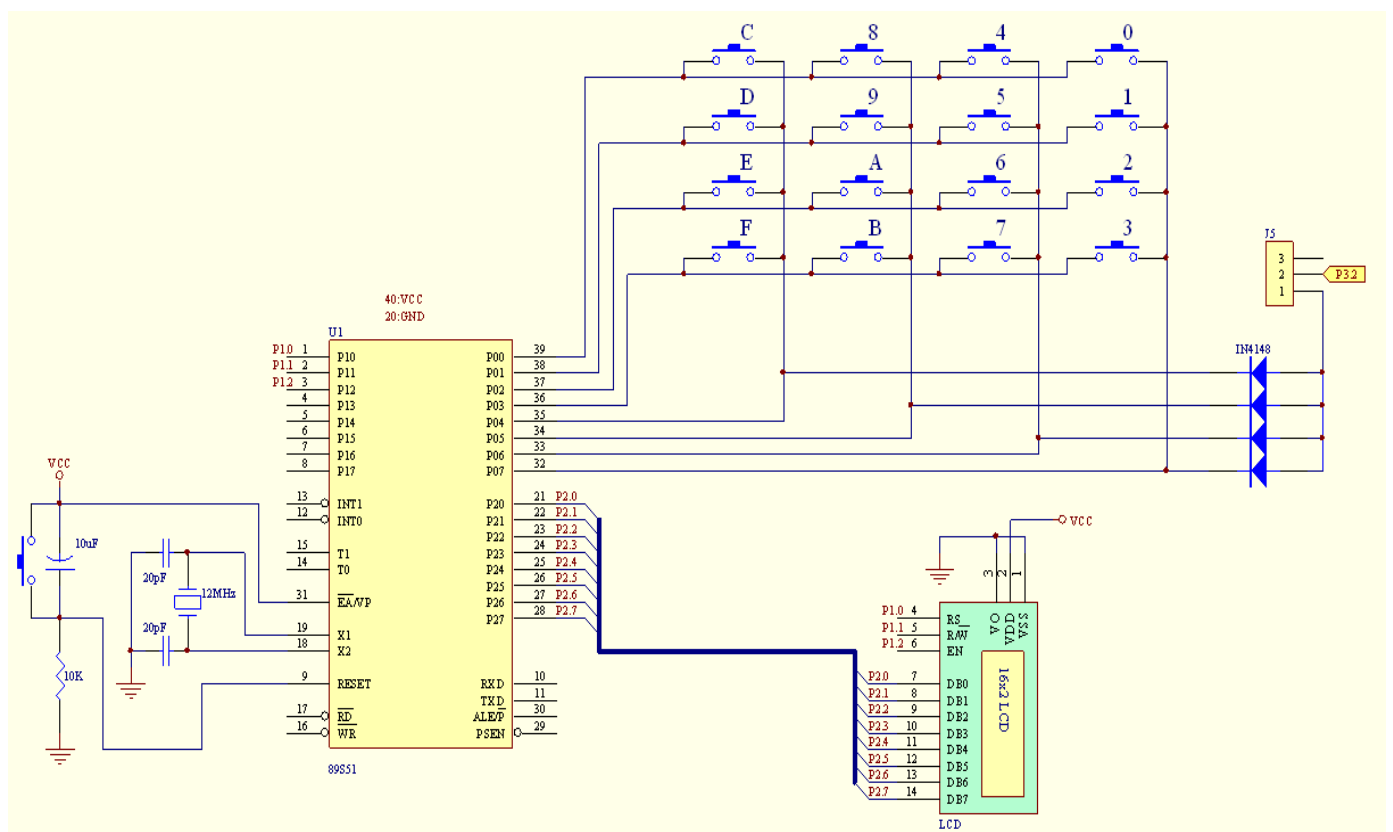


圖 5. 矩陣式鍵盤控制電路圖(外加二極體，由 P3.2 提供電流)

圖 3~圖 5 為常見的 3 種矩陣式鍵盤控制電路的接法，其中圖 3 只適用於 82G516，因為此顆晶片內部 P0 在高電位時有輸出電流，可確保按鍵沒有按下時讀回值為 1。由於標準 8051 的 P0 內部無提升電阻，如果採用此接法，按鍵沒有按下時讀回值易受干擾，比較容易誤動作，因此必須將按鍵改接到其他 IO 埠。圖 4 則為一種改良式接法，不用擔心接錯 IO 埠，但當 IO 埠內部有提升電阻時，這樣接實在是多此一舉，增加電路成本。圖 5 則是這張實驗板上的接法，透過 P3.2 提供按鍵所需之電流，看似複雜，這樣做最大好處是不用隨時做鍵盤掃描，甚至可是先讓 CPU 進入睡眠省電狀態，等有按鍵再喚醒 CPU。其作法是事先設定 P0.3~P0.0='0000'，P0.7~P0.4='1111'，並設定允許 INT1(P3.2)中斷，這時只要有任何一個鍵被壓下，都會使 P3.2=0 產生中斷，這時再透過程式掃描一次，看是哪一個鍵被壓下即可。

以下範例為一簡單的鍵盤測試程式，程式中包含掃描鍵盤，除彈跳及等按鍵放開三個主要部份，並將按鍵結果顯示於 P2 的 LED，由於 LED 只顯示最後一次的按鍵結果，所以除彈跳有無成功都看不出來，最好還是搭配 LCD 或 8 位數七段顯示器做練習，比較能看出成效。

按鍵測試練習

ASM

;鍵盤測試程式

```
KB          EQU P0
KB_BUF      E QU 30H

AGAIN:      MOV R4,#4
NEXT:      MOV KB_BUF,#11111110B          ;掃描訊號
           MOV KB,KB_BUF                 ;送至鍵盤
           MOV A,KB                       ;讀回鍵盤訊號
           ANL A,#11110000B               ;將後 4 位元清為 0
           CJNE A,#11110000B,RECHECK      ;判斷是否按鍵盤(前 4 位元有不為 0 的情況)
           MOV A,KB_BUF                   ;若無則將 KB_BUF 內的資料左旋
           RL A                            ;
           MOV KB_BUF,A                   ;準備掃描下一行
           DJNZ R4,NEXT                    ;R4 內的資料減一,判斷是否掃描 4 次
           JMP AGAIN                       ;重新開始掃描
RECHECK:    MOV R5,#3                     ;延遲,除彈跳
           CALL DELAY
           MOV A,KB                       ;重新讀回按鍵訊號
           ANL A,#11110000B
           CJNE A,#11110000B,KEY_IN       ;判斷是否按鍵盤
           JMP AGAIN                       ;若無則為彈跳訊號,重新開始掃描
KEY_IN:     MOV A,KB                       ;讀回鍵盤訊號
           MOV P2,A                       ;將按鍵值送到 P2 的 LED 顯示

WAIT:      MOV A,KB                       ;重新讀回按鍵訊號
           ANL A,#11110000B
           CJNE A,#11110000B,WAIT         ;判斷按鍵是否放開

           JMP AGAIN

DELAY:     MOV R6,#100
DEL:       MOV R7,#100
           DJNZ R7,$
           DJNZ R6,DEL
           DJNZ R5,DELAY
           RET

END
```

C-coe

```
#include <reg51.h>
#include <intrins.h>

#define RL(x)  _crol_(x,1);    // 重新定義 RL 左旋函式

void delay(int t) // 延遲函數開始
{
    int i, j;                // 宣告整數變數 i,j
    for (i = 0; i < t; i++)    // 計數 t 次,延遲 t x 1ms
        for (j = 0; j < 800; j++); // 計數 800 次,延遲 1ms
}

void main(void)
{
    unsigned char kb_buf, x, tmp;

    while(1) {
        kb_buf = 0xfe;
        for (x = 0; x < 4; x++) {
            P0 = kb_buf;
            tmp = P0;

            //將 P0.3~P0.0 清為 0, 判斷 P0.7~P0.4 是否皆為 1,如果不是,代表有按鍵被壓下
            while((tmp & 0xf0) != 0xf0)
            {
                delay(3);                //延遲, 判斷是否為彈跳訊號
                tmp = P0;
                if ((tmp & 0xf0) != 0xf0)    //如果 P0.7~P0.4 仍然有 0 的情況, 則表示非彈跳訊號
                    P2 = P0;                //將 P0 的按鍵值送到 P2 顯示

                do {
                    tmp = P0;                //讀回按鍵值
                } while((tmp & 0xf0) != 0xf0); //直到 P0.7~P0.4 皆為 1 為止(代表按鍵已放開)
            }
            kb_buf = RL(kb_buf);            //掃描資料左旋, 準備掃描下一行
        }
    }
}
```